

UNIVERSIDAD ESPÍRITU SANTO

FACULTAD DE INGENIERÍA

Carrera de Ingeniería en Computación

Diseño de Software (UCOM0310)

PROYECTO INTEGRADOR

FASE 3 - CÓDIGO REFACTORIZADO E INFORME FINAL

Sistema de Reservas de Canchas Deportivas

Documento final basado en el diseño UML de Fase 1

Integrantes	Hipólito Gabriel Aguirre Leiton
Docente	Victoria García Velásquez, MGP.
Período	PEL 3 - 2026
Fecha	30/07/2026

Tabla de contenido

1. Contexto
2. Evidencia de pruebas
3. Refactorizaciones realizadas
4. Cobertura de los cuatro niveles solicitados
5. Diagrama de clases actualizado
6. Evolución del diseño
7. Estado final
8. Comandos útiles de verificación

1. Contexto

Este informe corresponde a la Fase 3 del proyecto integrador de Diseño de Software. La Fase 1 definió el diseño UML del sistema mediante el enfoque arquitectónico 4+1, con las clases Cliente, Cancha, Horario, Reserva, ServicioReserva, RepositorioReserva y Notificador. La Fase 2 implementó una versión funcional mínima y documentó malos olores como método largo, lista de parámetros, código duplicado, obsesión por primitivos, números mágicos y clase con exceso de responsabilidades.

En esta fase se refactorizó el código manteniendo los casos de uso principales: crear reserva, consultar disponibilidad y cancelar reserva. El objetivo fue mejorar el diseño sin alterar el comportamiento observable del sistema. Después de cada refactorización se ejecutaron las pruebas unitarias y se creó un commit independiente.

2. Evidencia de pruebas

Comando usado después de cada refactorización:

```
python -m pytest
```

Resultado final: 7 pruebas aprobadas.

Las pruebas existentes cubren creación exitosa de reservas, rechazo de conflictos, consulta de disponibilidad, cancelación, reserva desconocida y correo inválido.

3. Refactorizaciones realizadas

No.	Commit	Nivel	Detalle
1	9530008	Métodos	Se extrajeron métodos privados de validación en ReservationService. Olor atendido: método largo y código duplicado. Resultado: los métodos públicos quedaron más pequeños y legibles.
2	a1fa5a6	Datos	Se introdujo ReservationRequest como objeto de datos. Olor atendido: lista larga de parámetros. Resultado: la creación puede recibir una estructura única y el método anterior se conserva como fachada.
3	c0079fe	Datos	Se introdujo TimeSlot para inicio, duración, fin y solapamiento. Olor atendido: obsesión por primitivos y cálculos repetidos de fechas. Resultado: la regla de solapamiento pasó del repositorio al dominio.
4	aeaca8f	Clases/objetos y condicionales	Se introdujo ReservationStatus y Reservation.cancel(). Olor atendido: strings mágicos y manipulación externa del estado. Resultado: la reserva controla su propia transición a cancelada.
5	d4adbc5	Clases/objetos	Se extrajo ReservationValidator. Olor atendido: clase de servicio con demasiadas responsabilidades. Resultado: las validaciones quedaron en una clase especializada e inyectable.
6	1cc9d63	Condicionales	Se agregó has_active_conflict() y Reservation.is_confirmed(). Olor atendido: condicionales compuestos y consultas poco expresivas. Resultado: el servicio expresa la regla como disponibilidad/conflicto y el repositorio encapsula el filtro.

4. Cobertura de los cuatro niveles solicitados

Nivel	Evidencia en el código
Métodos	create_reservation redujo su extensión al delegar validaciones y al separar el flujo basado en ReservationRequest.
Clases/objetos	ReservationValidator concentra validaciones. Reservation encapsula la regla de cancelación mediante cancel().
Datos	ReservationRequest agrupa datos de entrada. TimeSlot representa el horario y concentra end_at y overlaps().
Condicionales	Reservation.is_confirmed() y has_active_conflict() reemplazan comparaciones y filtros compuestos.

5. Diagrama de clases actualizado

El diagrama actualizado conserva el diseño de Fase 1 y muestra la evolución realizada durante la refactorización. Horario se implementa como TimeSlot, Reserva controla su estado, y ReservationValidator aparece como colaborador especializado del servicio.

Elemento	Responsabilidad actualizada
Client	Mantiene identidad, nombre y correo del usuario que realiza reservas.
Court	Representa la cancha reservable y su estado de disponibilidad.
TimeSlot	Objeto de valor equivalente a Horario; calcula fin y detecta solapamientos.
ReservationStatus	Enumera CONFIRMED y CANCELLED para evitar strings mágicos.
Reservation	Relaciona cliente, cancha y horario; encapsula cancelación e inspección de estado.
ReservationRequest	Agrupar los datos requeridos para crear una reserva.
ReservationService	Orquesta crear reserva, consultar disponibilidad y cancelar reserva.
InMemoryReservationRepository	Persiste reservas en memoria y consulta conflictos activos.
ReservationValidator	Valida datos de cliente, cancha, periodo, notas y motivo.
NotificationService	Simula el envío de notificaciones de confirmación y cancelación.

Archivos del diagrama: docs/diagrama_clases_actualizado_fase_3.svg y docs/diagrama_clases_actualizado_fase_3.puml.

6. Evolución del diseño

- Horario se implementa como TimeSlot, objeto de valor encargado de end_at y overlaps().
- Reserva aumentó su cohesión porque controla su estado mediante cancel().
- ServicioReserva mantiene el rol de orquestador, pero ya no concentra todas las validaciones.
- ReservationValidator se agregó como colaborador especializado.
- ReservationStatus reemplaza literales de texto para evitar errores por strings mágicos.
- ReservationRequest documenta los datos necesarios para crear una reserva y facilita extender el caso de uso.

7. Estado final

La implementación final mantiene los tres casos de uso de la Fase 1 y conserva las pruebas unitarias aprobadas. La historia de Git contiene seis commits independientes de refactorización, lo que permite revisar cada diff antes/después y comprobar que el sistema se mantuvo funcional durante todo el proceso.

8. Comandos útiles de verificación

```
git log --oneline -6
python -m pytest
```

1cc9d63 refactor: simplificar condicionales de disponibilidad
d4adbc5 refactor: extraer validador de reservas
aeaca8f refactor: encapsular cambios de estado de reserva
c0079fe refactor: introducir objeto de valor para horario
a1fa5a6 refactor: introducir objeto de solicitud de reserva
9530008 refactor: extraer metodos de validacion del servicio de reservas